PEAK Data Track — Week 6: Evaluating & Improving Models

Mentor: Doris
Level: Beginner friendly
Session: 14 September 2026

What you will learn

- Review Week 5: features (X), target (y), training and testing
- Measure accuracy, precision, recall and F1-score
- Read a confusion matrix
- Compare Logistic Regression with a Decision Tree
- Use 5-fold cross-validation
- Improve a model with `GridSearchCV`
- Write a short evidence-based conclusion

1. Recap: Week 4

A **feature** is an input used to make predictions. A **target** is what we want to predict. A classification model predicts a category.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score,
recall_score,
                             f1_score, ConfusionMatrixDisplay,
classification_report)

iris = load_iris(as_frame=True)
X = iris.data
y = (iris.target == 0).astype(int)  # 1 = setosa, 0 = not setosa
print('Dataset shape:', X.shape)
print('Class counts:')
print(y.value_counts().rename(index={0:'Not setosa', 1:'Setosa'}))
display(X.head())
```

```
Dataset shape: (150, 4)
Class counts:
target
Not setosa    100
Setosa         50
Name: count, dtype: int64
```


```
{"summary":"{\n  \"name\": \"display(X\",\n  \"rows\": 5,\n
\"fields\": [\n    {\n      \"column\": \"sepal length (cm)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.2073644135332772,\n        \"min\": 4.6,\n        \"max\": 5.1,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          4.9,\n
5.0,\n          4.7\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal width (cm)\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.2588435821108957,\n        \"min\":
3.0,\n        \"max\": 3.6,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          3.0,\n          3.6,\n          3.2\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"petal length (cm)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.07071067811865474,\n        \"min\": 1.3,\n        \"max\": 1.5,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          1.4,\n
1.3,\n          1.5\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal width (cm)\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.0,\n        \"min\": 0.2,\n
\"max\": 0.2,\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          0.2\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


2. Split the dataset

Train on 80% of the data and test on the remaining 20%. `stratify=y` keeps similar proportions of each class in both sets. We keep the test set untouched during tuning.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print('Training rows:', len(X_train))
print('Testing rows:', len(X_test))
```

```
Training rows: 120
Testing rows: 30
```

```
y_train.head(10)
```

```
83     0
6      1
```

```
104    0
105    0
69     0
46     1
75     0
33     1
35     1
17     1
Name: target, dtype: int64
```

3. Train two models

Logistic Regression learns a linear boundary between categories. **Decision Tree** makes a series of if/then decisions.

A pipeline scales inputs for Logistic Regression using training data only, avoiding data leakage.

```
models = {
    'Logistic Regression': make_pipeline(StandardScaler(),
LogisticRegression(max_iter=1000, random_state=42)),
    'Decision Tree': DecisionTreeClassifier(max_depth=3,
random_state=42)
}
for name, model in models.items():
    model.fit(X_train, y_train)
    print(name, 'trained successfully')
```

```
Logistic Regression trained successfully
Decision Tree trained successfully
```

4. Understand evaluation metrics

- **Accuracy:** What fraction of all predictions were correct?
- **Precision:** Of flowers predicted as setosa, how many really were setosa?
- **Recall:** Of all real setosa flowers, how many did we identify?
- **F1-score:** A balance between precision and recall.

Important: Accuracy can be misleading when one category is much more common. The best metric depends on the problem. For example, in fraud detection, missing actual fraud may be especially costly.

```
results = []
for name, model in models.items():
    predicted = model.predict(X_test)
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, predicted),
        'Precision': precision_score(y_test, predicted,
zero_division=0),
```

```
        'Recall': recall_score(y_test, predicted, zero_division=0),
        'F1-score': f1_score(y_test, predicted, zero_division=0)
    })
comparison = pd.DataFrame(results).set_index('Model')
display(comparison.round(3))
```


```
{"summary":"{\n  \"name\": \"display(comparison\",\n  \"rows\": 2,\n
\"fields\": [\n    {\n      \"column\": \"Model\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n
\"Decision Tree\",\n          \"Logistic Regression\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Accuracy\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 0.0,\n
\"min\": 1.0,\n        \"max\": 1.0,\n        \"num_unique_values\":
1,\n        \"samples\": [\n          1.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Precision\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0,\n        \"min\": 1.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          1.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Recall\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 0.0,\n
\"min\": 1.0,\n        \"max\": 1.0,\n        \"num_unique_values\":
1,\n        \"samples\": [\n          1.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"F1-score\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 0.0,\n
\"min\": 1.0,\n        \"max\": 1.0,\n        \"num_unique_values\":
1,\n        \"samples\": [\n          1.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


Read the confusion matrix

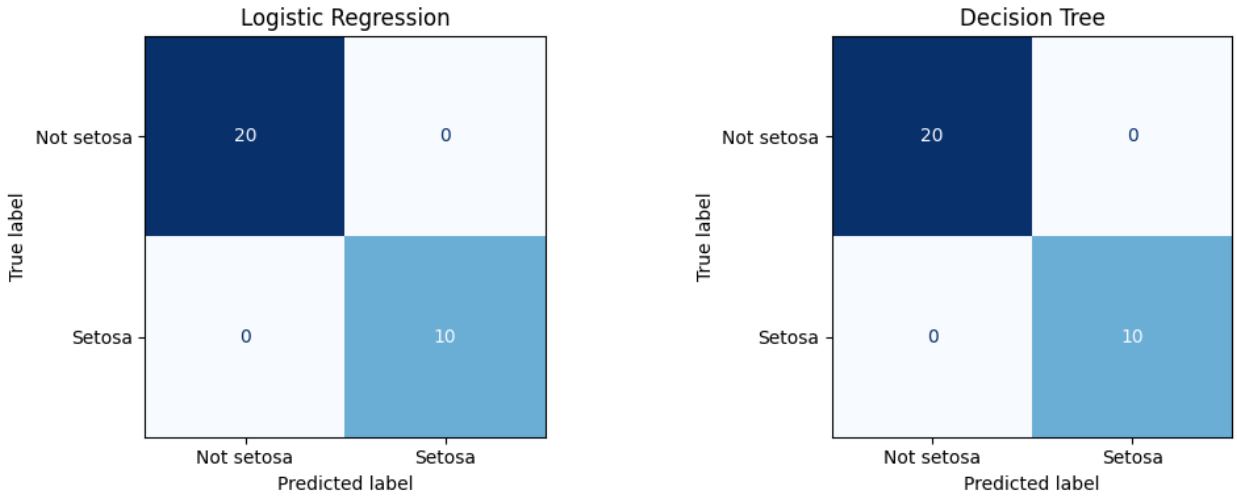
The four cells represent true negatives, false positives, false negatives and true positives. Look for errors away from the main diagonal.

```
X_test.head(10)
```


```
{"summary":"{\n  \"name\": \"X_test\",\n  \"rows\": 30,\n  \"fields\":
[\n    {\n      \"column\": \"sepal length (cm)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.7442113777097495,\n        \"min\": 4.4,\n        \"max\": 7.1,\n
\"num_unique_values\": 18,\n        \"samples\": [\n          6.4,\n
5.2,\n          4.5\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal width (cm)\",\n      \"properties\": {\n        \"dtype\":
```

\"number\",\n   \"std\": 0.42724727019029324,\n   \"min\":
2.3,\n   \"max\": 4.1,\n   \"num_unique_values\": 15,\n
\"samples\": [\n   2.9,\n   2.5,\n   3.1\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"petal length (cm)\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
1.7781690182519951,\n   \"min\": 1.3,\n   \"max\": 5.9,\n
\"num_unique_values\": 17,\n   \"samples\": [\n   5.5,\n
1.5,\n   4.5\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"petal width (cm)\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.8007252459740669,\n   \"min\":
0.1,\n   \"max\": 2.4,\n   \"num_unique_values\": 16,\n
\"samples\": [\n   1.8,\n   0.2,\n   1.4\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   }\n]\n}","type":"dataframe","variable_name":"X_test"}

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, (name, model) in zip(axes, models.items()):
    ConfusionMatrixDisplay.from_estimator(model, X_test, y_test,
        display_labels=['Not setosa', 'Setosa'], ax=ax, cmap='Blues',
colorbar=False)
    ax.set_title(name)
plt.tight_layout()
plt.show()
```



Your turn

1. Which model has the higher F1-score?
2. Are any real setosa flowers incorrectly classified?
3. Why might we consider recall instead of accuracy in some real-world problems?

5. Cross-validation: test consistency

Instead of relying on one training split, **5-fold cross-validation** divides the training data into five parts. Each part becomes a validation fold once. We calculate the average F1-score across the five runs.

Rule: Cross-validation happens on the training data only. The held-out test data is for the final evaluation.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_results = []
for name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=cv,
scoring='f1')
    cv_results.append({'Model': name, 'Mean CV F1': scores.mean(), 'CV
Std Dev': scores.std()})
    print(f'{name}: fold F1-scores = {scores.round(3)}')
cv_table = pd.DataFrame(cv_results).set_index('Model')
display(cv_table.round(3))
```

```
Logistic Regression: fold F1-scores = [1. 1. 1. 1. 1.]
Decision Tree: fold F1-scores = [1. 1. 1. 1. 1.]
```

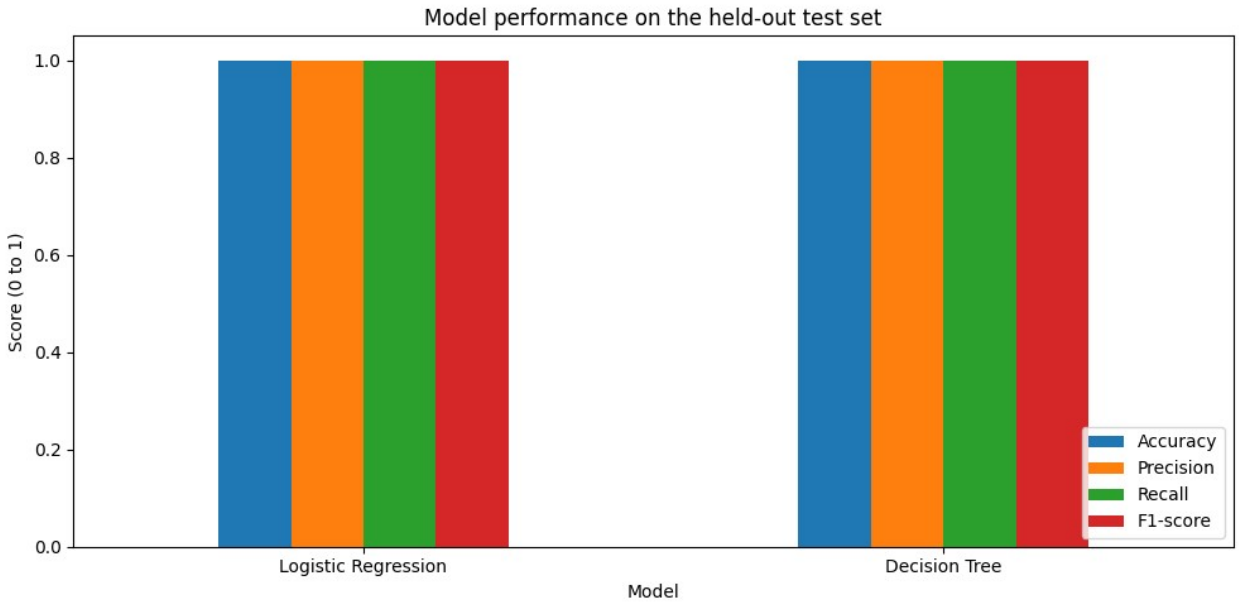

```
{"summary":"{\n  \"name\": \"display(cv_table\",\n  \"rows\": 2,\n
\"fields\": [\n    {\n      \"column\": \"Model\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n
\"Decision Tree\",\n          \"Logistic Regression\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Mean CV F1\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0,\n        \"min\": 1.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          1.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"CV Std Dev\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0,\n        \"min\": 0.0,\n        \"max\": 0.0,\n
\"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe"}
```


Interpretation: A higher mean cross-validation score suggests better validation performance. A smaller standard deviation suggests more consistent results across these folds. With a small, easy dataset, models may tie; that is a valid outcome.

6. Compare models visually

```
comparison[['Accuracy', 'Precision', 'Recall', 'F1-score']].plot(
    kind='bar', figsize=(10, 5), ylim=(0, 1.05), rot=0
)
```

```
plt.title('Model performance on the held-out test set')
plt.ylabel('Score (0 to 1)')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```



7. Basic hyperparameter tuning

A **hyperparameter** is a setting chosen before training. For a Decision Tree, `max_depth` controls how many levels the tree can grow. `GridSearchCV` tries different settings using cross-validation on training data. It selects the setting with the best average validation F1-score.

```
param_grid = {
    'max_depth': [1, 2, 3, 4, None],
    'min_samples_leaf': [1, 2, 4]
}
search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    scoring='f1',
    cv=cv,
    n_jobs=-1
)
search.fit(X_train, y_train)
print('Best settings:', search.best_params_)
print('Best cross-validation F1:', round(search.best_score_, 3))

tuned_tree = search.best_estimator_
tuned_predictions = tuned_tree.predict(X_test)
print('Tuned tree test F1:', round(f1_score(y_test, tuned_predictions,
```

```
zero_division=0), 3))
print(classification_report(y_test, tuned_predictions,
    target_names=['Not setosa', 'Setosa'], zero_division=0))

Best settings: {'max_depth': 1, 'min_samples_leaf': 1}
Best cross-validation F1: 1.0
Tuned tree test F1: 1.0
              precision    recall  f1-score   support

  Not setosa       1.00      1.00      1.00        20
      Setosa       1.00      1.00      1.00        10

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Did tuning help?

Compare the tuned tree's test F1 with the original Decision Tree's test F1. Tuning **does not guarantee** an improvement, especially on a small or easy dataset. Avoid repeatedly changing settings based on the test set.

```
before = comparison.loc['Decision Tree', 'F1-score']
after = f1_score(y_test, tuned_predictions, zero_division=0)
print(f'Original Decision Tree test F1: {before:.3f}')
print(f'Tuned Decision Tree test F1:    {after:.3f}')
print(f'Change: {after - before:+.3f}')

Original Decision Tree test F1: 1.000
Tuned Decision Tree test F1:    1.000
Change: +0.000
```

8. Interpretation and conclusion

Complete these sentences after running the code:

1. The model with the highest **mean cross-validation F1** was _____.
2. Its average score was _____.
3. On the held-out test set, the model(s) with the highest F1-score was/were _____.
4. The most important metric for this example is _____ because _____.
5. Tuning changed the Decision Tree's test F1 from _____ to _____.
6. **Conclusion:** I would choose _____ because _____.

If the models tie, say so. You can then discuss simplicity, interpretability, speed, and stability.

---

9. Week 6 assignment — submit your notebook

Due: 3 October 2026

Use your **Week 5 model and dataset** where possible (not necessarily the Iris example).

Tasks

1. Briefly describe your dataset, features, target and whether the task is classification or regression.
2. Evaluate your Week 5 model using **at least two suitable metrics**. Classification: accuracy, precision, recall, F1. Regression: MAE, RMSE, R^2.
3. Train a **second model** on the same training data.
4. Show a comparison table with metrics for both models on the same held-out test set.
5. Use **cross-validation on training data** and report the average score.
6. Tune at least one hyperparameter for one model using cross-validation.
7. Write **4–6 sentences** explaining which model you prefer and why. A tie is acceptable if supported by evidence.

Submission checklist

- ☐ Notebook runs from top to bottom
- ☐ At least two evaluation metrics
- ☐ Two models compared fairly
- ☐ Cross-validation results included
- ☐ Hyperparameter tuning demonstrated
- ☐ Final conclusion written in your own words

Submission: Download your Colab notebook via **File → Download → Download .ipynb**.

Extra challenge (optional)

Try an imbalanced dataset. Compare accuracy and recall. Explain why they tell different stories.